

IV. РЕАЛИЗАЦИЯ НА ЗАДАЧИ СЪС СЛОЖНИ МАТЕМАТИЧЕСКИ ИЗЧИСЛЕНИЯ

Цел на упражнението

Това упражнение има за цел да представи реализации на задачи със сложни математически изчисления, с прилагане на масиви и ASCII кодове. То демонстрира използване на контроли със списъчна структура, като listBox, checkedListBox и създаване на приложения чрез контрола WebBrowser.

1. Масив

2.Едномерен масив

2.1. Деклариране на едномерен масив

Масиви в C# се декларират по следния начин: `int[] myArray;`

2.2. Създаване (заделяне) на масив – оператор new

В C# масив се създава с ключовата дума **new**, която служи за заделяне (алокиране) на памет: `int[] myArray = new int[6];`

2.3. Инициализация на едномерен масив

```
int[] myArray = { 1, 2, 3, 4, 5, 6 };
```

При този синтаксис фигурните скоби заместват оператора new и между тях са изброени началните стойности на масива, разделени със запетаи. Техният брой определя дължината му.

2.4. Достъп до елементите на едномерен масив

```
int[] myArray = new int[6];  
myArray[index] = 100;  
myArray[1] = 1;  
myArray[5] = 5;
```

3. Многомерен масив

Освен едномерните масиви, известни в математиката като "вектори", в практиката, често се ползват масиви с повече от едно измерение. Те се наричат многомерни. Всеки допустим в C# тип може да бъде използван за тип на елементите на масив. Масивите също може да се разглеждат като допустим тип. Така може да има и масив от масиви. Масив с две измерения или още матрица се нарича двумерен. Масив с повече от едно измерение се нарича многомерен. [4]

3.1. Деклариране и заделяне на многомерен масив

Многомерните масиви се декларират по начин аналогичен на едномерните. Всяка тяхна размерност (освен първата) се означава със запетая: `int[,] intMatrix; float[,] floatMatrix; string[,] strCube;`

3.2. Инициализация на двумерен масив

Инициализацията на многомерни масиви е аналогична на инициализацията на едномерните. Стойностите на елементите могат да се изброяват непосредствено след декларацията:

```
int[,] matrix = { {1, 2, 3, 4}, // row 0 values  
{5, 6, 7, 8}, // row 1 values };
```

3.3. Достъп до елементите на многомерен масив

Матриците имат две размерности и съответно всеки техен елемент се достъпва с помощта на два индекса – един за редовете и един за колоните. Многомерните масиви имат различен индекс за всяка размерност. Масивът matrix има 8 елемента, разположени в 2 реда и 4 колони. Всеки елемент може да се достъпи по следния начин: `matrix[0, 0] matrix[0, 1] matrix[0, 2] matrix[0, 3]`

```
matrix[1, 0] matrix[1, 1] matrix[1, 2] matrix[1, 3]
```

Ако се означа индекса по редове с row, а индекса по колони с col, тогава достъпът до елемент от двумерен масив има следния общ вид: `matrix[row, col]`

При многомерните масиви всеки елемент се идентифицира уникално с толкова на брой индекси, колкото е размерността на масива: `nDimensionalArray[index1, ... , indexN]`. [4]

3.4. Дължина на многомерен масив

Всяка размерност на многомерен масив има собствена дължина, която е достъпна по време на изпълнение на програмата.

В примера за двумерен масив:

`int[,] matrix = { {1, 2, 3, 4}, {5, 6, 7, 8} };` може да се извлече броят на редовете на този двумерен масив чрез `matrix.GetLength(0)`, а дължината на всеки от редовете (т.е. броят колони) с `matrix.GetLength(1)`.

4. Масив от масиви

В C# може да се използва масив от масиви или така наречените назъбени (jagged) масиви. Назъбените масиви представляват масиви от масиви или по-точно масиви, в които един ред на практика е също масив и може да има различна дължина от останалите в назъбения масив. [4]

4.1. Деклариране и заделяне на масив от масиви

Единственото по-особено при назъбените масиви е, че няма само една двойка скоби, както при обикновените масиви, а има по една двойка скоби за всяко от измеренията. Заделянето става по същия начин: `int[][] jaggedArray; jaggedArray = new int[2][];`
`jaggedArray[0] = new int[5]; jaggedArray[1] = new int[3];`

Ето как се декларира, заделя и инициализира един масив от масиви:

```
int[][] myJaggedArray =  
{ new int[] {5, 7, 2}, new int[] {10, 20, 40}, new int[] {3, 25} };
```

4.2. Инициализиране и достъп до елементите на масив от масиви

Достъпът до елементите на масивите, които са част от назъбения, отново се извършва по индекса им. Пример за достъп до елемента с индекс 3 от масива, който се намира на индекс 0 в по-горе дефинирания назъбен масив `jaggedArray`:

```
myJaggedArray[0][3] = 45;
```

Елементите на назъбения масив може да са както едномерни масиви, така и многомерни такива. Ето един пример за назъбен масив от двумерни масиви:

```
int[,] jaggedOfMulti = new int[2][,];  
jaggedOfMulti[0] = new int[,] { { 5, 15 }, { 125, 206 } };  
jaggedOfMulti[1] = new int[,] { { 3, 4, 5 }, { 7, 8, 9 } }; [4]
```

5. Задачи за изпълнение

Задача 1. Като използвате познанията си за палиндром, създайте приложение, което при зададен низ, определя дали той е палиндром или не.



Фиг.4.1. Интерфейс на приложение Палиндром, в режим проектиране

Забележка:

Палиндром е една или няколко думи или числа, които се четат по един и същ начин от ляво надясно и от дясно наляво.

Примерно решение:

```
public class StringHelper {  
    public static string ReverseString(string s){  
        char[] arr = s.ToCharArray();  
        Array.Reverse(arr);  
        return new string(arr);  
    }  
    private void button1_Click(object sender, EventArgs e) {  
        string s1, s2;  
        s1 = s2 = textBox1.Text;  
        s1 = StringHelper.ReverseString(s1);  
        if (s1 == s2)  
            MessageBox.Show("Да. Палиндром е!");  
        else  
            MessageBox.Show("Не е палиндром!");  
    }  
}
```

Задача 2. Да се напише програма, която при зададен символ, връща неговото съответствие от ASCII таблица.

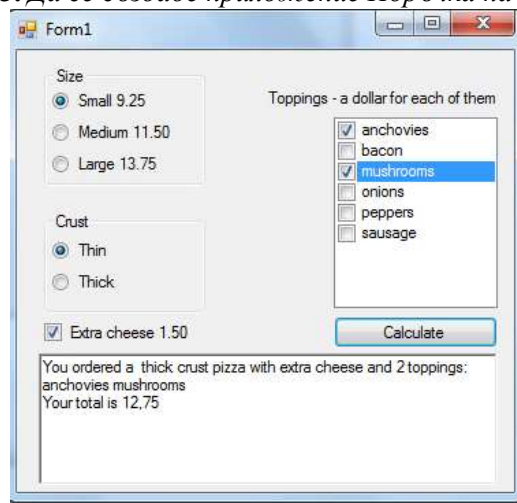


Фиг.4.2. Интерфейс на приложение ASCII код, в режим проектиране

Частично решение:

```
private void button1_Click(object sender, EventArgs e){
    string name;
    int letter;
    name = textBox1.Text;
    for (int index = 0; index < name.Length; index++){
        letter = name[index];
        listBox1.Items.Add(letter + ""); } }
```

Задача 3. Да се създаде приложение Поръчка на пица. (фиг.4.3.)



Фиг.4.3. Интерфейс на приложение Поръчка на пица, в режим изпълнение

Частично решение:

За прочитане на данните от текстов файл:

```
1) string[] lines = System.IO.File.ReadAllLines("d:\\Toppings.txt");
    foreach (string str in lines) {
        checkedListBox1.Items.Add(str); }
```

За достъп до всеки избран елемент в checkedListBox и извеждане на общия брой маркирани с чек елементи:

```
2) private void button1_Click(object sender, EventArgs e) {
    string s="Checked items:\n"; int br = 0;
```

```
    for (int x = 0; x < checkedListBox1.Items.Count; x++)
    {
        if (checkedListBox1.GetItemChecked(x))
        {
            br++;
            s = s + "Item " + (x + 1).ToString() + " = " +
                checkedListBox1.Items[x].ToString() + "\n";
        }
    }
```

```
    richTextBox1.Text = s + " Total br=" + br.ToString(); }
```

```
3) private void button2_Click(object sender, EventArgs e) {
    if (checkedListBox1.CheckedItems.Count != 0){
        //Ако има отметнати елементи, резултата да се отпечата
        string s = "";
        for (int x = 0; x < checkedListBox1.CheckedItems.Count; x++){
            s = s + "Checked Item " + (x + 1).ToString() + " = " +
                checkedListBox1.CheckedItems[x].ToString() + "\n"; }
        MessageBox.Show(s); }}
```

Задача 4. Да се създаде приложение *Web Browser*.



Фиг.4.4. Интерфейс на приложение *Web Browser*,
в режим изпълнение

За реализиране на приложението да се добавят контролите: *WebBrowser*, един *textBox* и пет бутона с надписи: *button1* – Back, *button2* – Forward, *button3* – Refresh, *button4* – Stop, *button5* – Go.

Примерно решение:

```
private void button2_Click(object sender, EventArgs e) {  
    webBrowser1.GoForward(); }  
private void webBrowser1_DocumentCompleted(object sender, webBrowserDocumentCompletedEventArgs e)  
{  
    textBox1.Text = e.Url.ToString(); }  
private void button5_Click(object sender, EventArgs e) {  
    webBrowser1.Navigate(textBox1.Text); }  
private void button1_Click(object sender, EventArgs e) {  
    webBrowser1.GoBack(); }  
private void button3_Click(object sender, EventArgs e) {  
    webBrowser1.Refresh(); }  
private void button4_Click(object sender, EventArgs e) {  
    webBrowser1.Stop(); }
```

6.Задачи за самостоятелна работа

Задача 1. По дадено число N да се генерира и отпечата квадратна матрица, съдържаща числата от 0 до N , разположени като спирала, започваща от центъра на матрицата и движеща се обратно на часовниковата стрелка.

Задача 2. Дадена е мрежа от $m \times n$ квадратчета, като всяко квадратче е с определен цвят – бял или черен. Път се нарича редица от съседни във вертикално или хоризонтално направление квадратчета с един и същ цвят. Област се нарича множество от квадратчета с един и същ цвят между всеки две, от които има път. При зададено квадратче да се определи: 1) броят на квадратчетата от областта, в която се съдържа даденото квадратче; 2) броят на областите с цвят, съвпадащ с цвета на даденото квадратче; 3) броят на областите с цвят, различен от цвета на даденото квадратче; 4) броят на квадратчетата с цвят, съвпадащ с цвета на даденото квадратче, които не са в една област с него.